JJBike

1. Objective.

Build a data warehouse so that the administrator of the fictional company JJBike can understand their business.

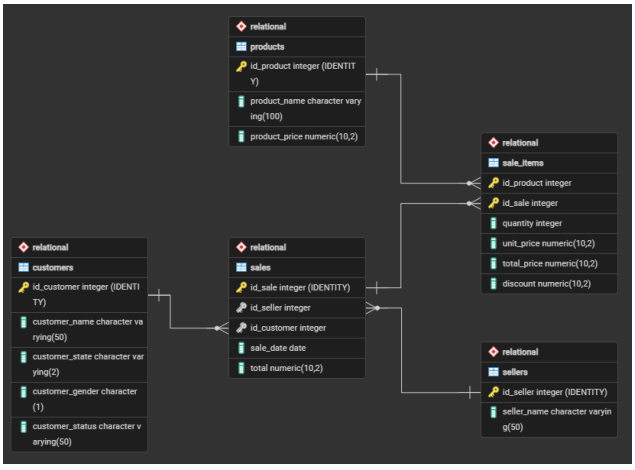
2. Modeling.

To create the analytical database structure model (OLAP) considering the transactional database (OLTP), the subject to be analyzed (the fact), and the dimensions (the various viewpoints from which to analyze the fact (the dimensions)), using the Star Schema model (the most common model for Business Intelligence), the following transformations were performed:

Dimension	Attributes	Source Table
dim_product (what?)	product_key (SK)	-
	id_product	product
	product_name	product
	validity_start_date (versioning)	-
	validity_end_date (versioning)	-
dim_seller (who?)	seller_key (SK)	-
	id_seller	seller
	seller_name	seller
	validity_start_date (versioning)	-
	validity_end_date (versioning)	-
dim_customer (for whom?)	customer_key (SK)	-
	id_customer	customer
	customer_name	customer
	customer_state	customer
	customer_gender	customer
	customer_status	customer
	validity_start_date (versioning)	-
	validity_end_date (versioning)	-
dim_time (when?)	time_key (SK)	-

	date	-
	time_day	-
	time_month	-
	time_year	-
	time_weekday	-
	time_quarter	-

Fact	Consolidated Table	Granularity	Attributes	Source Table
fact_sales	sales	Item sold.	sale_key (SK)	-
			seller_key (FK)	dim_seller
			customer_key (FK)	dim_customer
			product_key	dim_product
			time_key (FK)	dim_time
			quantity	sale_items
			unit_price	sale_items
			total_price	sale_items
			discount	sale_items



3. pgAdmin 4.

In pgAdmin 4, the database 'JJBIKE' was created.

- Note: The Snake Case naming convention will be used in the SQL code development.

4. Transactional Environment (OLTP) – pgAdmin 4.

4.1. Creation of the relational schema.

The Relational schema was created, serving as the source layer for the JJBike company's transactional data. Within it, the file 1. Scripts_Create_Table_Relational.sql created five operational tables. In all of them, the primary key is automatically generated by the GENERATED ALWAYS AS IDENTITY clause, eliminating the need to control external sequences. All columns, except the primary keys themselves, were defined with NOT NULL, ensuring that no record is persisted with mandatory data open.

- **sellers:** Stores the sellers' data. The seller_name attribute identifies the seller's name and is the only field besides the PK, reflecting the intentional scope of this entity in the transactional model;
- **products:** Stores the product catalog. The product_name attribute identifies the product and product_price records the current list price at the time of registration. Both are mandatory, as a product without a name or price cannot participate in a sales process; customers: Stores customer registration information. The customer_name attribute identifies the customer; customer_state records the state of origin (Varchar(2)); customer_gender records the gender in a single character (Char(1)); and customer_status records the customer's plan level, an attribute that, because it changes over time, will be the main element tracked by SCD Type 2 in the analytical environment;
- **sales:** Represents the header of each sale. The id_seller attribute is a foreign key that references Relational.sellers, and id_customer is a foreign key that references Relational.customers, both mandatory, as a sale cannot exist without an associated salesperson and customer. The sale_date attribute records the date the transaction occurred, and total stores the consolidated value of the sale;
- **sale_items:** Linking table between sales and products, with a primary key composed of id_product and id_sale. Two distinct exclusion policies were applied to preserve referential integrity in opposite directions:
    - ON DELETE RESTRICT on id_product: blocks the deletion of a record in Relational.products while it is referenced in some sales item, preventing historical data from losing the reference to the transacted product;
    - ON DELETE CASCADE on id_sale: when a record is deleted in Relational.sales, all corresponding items in sale_items are automatically removed, preventing orphaned items from a non-existent sale;
    - The attributes quantity, unit_price, total_price, and discount are the detail metrics of each item and were declared NOT NULL because they will directly feed the measure columns of the fact table in the analytical environment.

4.2. Populating the tables

The data was inserted using the files 2. Insert_Into_Customers.sql, 3. Insert_Into_Products.sql, 4. Insert_Into_Sellers.sql, 5. Insert_Into_Sales.sql, and 6. Insert_Into_SaleItems.sql. Each file executes INSERT INTO commands on the respective table in the Relational schema, specifying only the data columns; primary keys are omitted because the GENERATED ALWAYS AS IDENTITY mechanism automatically and sequentially assigns them to each inserted record.

5. Staging Environment (invisible/conceptual).

There was no processing or cleaning of raw data before creating the dimensions; the direct data transfer between PostgreSQL schemas itself acted as the load trigger.

6. Analytical Environment (OLAP) – pgAdmin 4.

6.1. Creation of the schema and dimensional tables.

The Dimensional schema was created, representing the Data Warehouse structured in a Star Schema model. The file 1. Scripts_Create_Table_Dimensional.sql created the dimensions dim_seller, dim_customer, dim_product, and dim_time, in addition to the fact table fact_sales. In all tables, the Surrogate Key (SK) is automatically generated by GENERATED ALWAYS AS IDENTITY, serving as the internal primary key of the DW, decoupled from the source IDs of the transactional system.

All columns were defined with NOT NULL, with the sole exception of validity_end_date in dimensions with SCD Type 2. This field remains NULL while the record is active; it is only filled with the current date when an attribute change is detected and a new version of the record needs to be created.

The dimensional tables have the following attribute structure:

- **dim_seller:** seller_key is the automatically generated SK and acts as the internal PK of the DW. id_seller preserves the source ID from Relational.sellers, maintaining traceability between the two environments. seller_name carries the seller's name. validity_start_date receives current_date at the time the record is inserted, marking the beginning of its validity. validity_end_date remains NULL while the record is the active version;
- **dim_customer:** customer_key is the SK, the internal PK of the DW. id_customer preserves the source ID. customer_name, customer_state, customer_gender, and customer_status are the tracked descriptive attributes; any change in any of them triggers SCD Type 2 versioning. validity_start_date and validity_end_date control the validity period of each version of the record;
- **dim_product:** product_key is the SK. id_product preserves the source ID. product_name is the only descriptive attribute tracked; a change in its value triggers versioning. validity_start_date and validity_end_date function the same way as in other SCD Type 2 dimensions;
- **dim_time:** time_key is the SK. time_date stores the complete date and serves as a linking field with the transactional system during fact loading. time_day, time_month, time_year, and time_quarter are derived from time_date and enable analytical groupings by temporal granularity. time_weekday stores the day of the week as an integer (0 = Sunday, 6 = Saturday). This dimension does not have validity fields because time is immutable;
- **fact_sales:** sale_key is the SK of the fact. seller_key, customer_key, product_key, and time_key are the FKs that reference the respective dimensions, all declared NOT NULL, because each fact must be associated with all dimensions of the model. quantity, unit_price, total_price, and discount are the metrics for each item sold, also NOT NULL.

6.2. Population of the time dimension.

The dim_time dimension was populated by file 2. Insert_Into_dim_time.sql. The strategy adopted was the programmatic generation of a continuous date range from January 1, 1970 to December 31, 2049, covering 29,220 records (indices 0 to 29,219). The script is composed of three chained blocks:

6.3. Extraction (E) and loading (L) of data from the transactional environment (OLTP) to the analytical environment (OLAP).

The loads were performed by file 3. Script.sql, structured in sequential phases to demonstrate the complete operation of the Type 2 SCD.

6.3.1. Initial loading of dimensions.

The same double CTE pattern was applied to the three dimensions supporting Type 2 SCD. The mechanism operates in three chained steps.

- **CTE S:** Selects all records from the source table in the Relational schema, for example, Relational.customers. This CTE represents the current state of the transactional source and serves as a basis for comparison for the following steps;
- **CTE UPD:** Executes an UPDATE on the target dimension, for example, Dimensional.dim_customer. The filter T.id_customer = S.id_customer AND T.validity_end_date IS NULL locates, for each source ID, the only record still active in the dimension, the one whose validity_end_date is still NULL. A second condition checks if any tracked attribute differs between the source and the dimension (customer_name, customer_state, customer_gender, or customer_status). When a difference is detected, validity_end_date receives current_date, ending the validity of that version of the record. The RETURNING T.id_customer clause returns the IDs of the newly closed records for use in the next step;
- **INSERT final:** Inserts new records into the dimension. The WHERE condition captures two groups: (1) the IDs returned by the CTE UPD, which have had a version terminated and need a new active entry, and (2) IDs that do not yet exist in the dimension, completely new records. In both cases, validity_start_date receives current_date, marking the start of the validity of the new version, and validity_end_date receives NULL, indicating that this is the currently valid record.

6.3.2. Loading the fact table (month by month).

The fact_sales table is populated by an INSERT INTO ... SELECT that consolidates data from the Relational schema with the dimensions of the Dimensional schema. Sales were loaded month by month, January, February, March, and subsequent months, to enable simulation and verification of the Type 2 SCD between loads.

The SELECT block operates with the following attribute and key mapping:

- **Metrics:** quantity, unit_price, total_price, and discount are extracted directly from Relational.sale_items, as they represent the detail data of each item sold, the granularity of the fact table;
- **seller_key:** Obtained by INNER JOIN between Relational.sales and Dimensional.dim_seller using v.id_seller = vdd.id_seller AND vdd.validity_end_date IS NULL. The `validity_end_date IS NULL` filter ensures that the fact record registers the SK of the vendor version that was active at the time of loading, and not of a closed historical version;
- **customer_key:** Obtained by INNER JOIN between `Relational.sales` and `Dimensional.dim_customer` using `v.id_customer = c.id_customer AND

c.validity_end_date IS NULL`. The `validity_end_date IS NULL` filter is the central mechanism of SCD Type 2 in loading the fact record: it ensures that the registered SK points to the current `customer_status` at the time that month was loaded, and not to previous or subsequent versions of the customer;

- **product_key:** Obtained by INNER JOIN between `Relational.sale_items` and `Dimensional.dim_product` using `iv.id_product = p.id_product AND p.validity_end_date IS NULL`. The same `validity_end_date IS NULL` filter captures only the active version of the product at the time of loading;
- **time_key:** Obtained by the INNER JOIN between `Relational.sales` and `Dimensional.dim_time` using `v.sale_date = t.time_date`. The `sale_date` field from the transactional table is directly compared to the `time_date` field of the dimension, locating the record that represents exactly the day the sale occurred. Since each date has exactly one record in `dim_time`, there is no risk of duplication in this join;
- **Temporal filter:** The function `date_part('MONTH', v.sale_date)` extracts the month number from `sale_date` and restricts the load to sales from the desired period, = 01 for January, = 02 for February, = 03 for March and > 03 for the remaining months.

6.3.3. Simulation of status change and history processing.

To demonstrate SCD Type 2 versioning, two direct updates were applied to Relational.customers.

- **After the January load:** UPDATE Relational.customers SET customer_status = 'Gold' WHERE id_customer BETWEEN 1 AND 5. The customer_status attribute of customers with IDs 1 to 5 was promoted to 'Gold';
- **After the March load:** UPDATE Relational.customers SET customer_status = 'Platinum' WHERE id_customer = 3. The customer_status attribute of customer with ID 3 was promoted to 'Platinum';
- With each change, the load block of dimensions (CTE S + CTE UPD + INSERT) was re-executed. The mechanism detects the discrepancy in customer_status, closes the previous record by filling validity_end_date with current_date and inserts a new line with validity_start_date equal to current_date and validity_end_date equal to NULL, preserving the complete version history of each customer.

6.3.4. Verification and auditing of SCD Type 2.

Throughout the roadmap, audit queries were executed to validate the behavior of SCD Type 2. The SELECTs cross fact_sales with dim_customer, and when necessary with dim_time, via INNER JOIN by the respective SKs, filtering c.id_customer BETWEEN 1 AND 5. The points below show what the queries demonstrate.

- The January records in fact_sales point to the old SKs of customers 1 to 5, that is, to the version in which customer_status was not yet 'Gold'. The historical link is preserved in the fact sheet even after the update in the dimension;
- The records from February onwards point to the new SKs, reflecting the customer_status in effect at the time each month was loaded;
- An additional query with filter t.time_month = 3 confirms that none of the customers with IDs 1 to 5 made purchases in March, validating that the fact sheet did not generate records for that group during that period.

6.4. Cube.

Since pgAdmin 4 is not a native OLAP server, the denormalized table Dimensional.sales_cube was created using the 4.Cube.sql file. It joins the fact table to all dimensions in a single flat structure, prepared for direct consumption by Power BI.

- The SELECT block selects only the descriptive attributes and metrics relevant for analysis. The SKs (seller_key, customer_key, product_key, time_key), the source IDs, and the validity fields (validity_start_date, validity_end_date) are deliberately excluded; they are internal technical control keys with no analytical value. The selected attributes are: customer_name, customer_state, customer_gender, and customer_status from dim_customer; product_name from dim_product; time_date, time_day, time_month, time_year, and time_quarter from dim_time; seller_name from dim_seller; and the metrics quantity, unit_price, total_price, and discount from fact_sales;
- The INTO Dimensional.sales_cube block physically creates the target table in the Dimensional schema, consolidating all the data needed for analysis in Power BI into a single structure;
- The FROM block starts from Dimensional.fact_sales fv and connects all dimensions using INNER JOIN: dim_customer dc via fv.customer_key = dc.customer_key; dim_product dp via fv.product_key = dp.product_key; dim_time dt via fv.time_key = dt.time_key; and dim_seller dv via fv.seller_key = dv.seller_key. The use of INNER JOIN in all joins ensures that only matches across all dimensions make up the cube; any record without an associated dimension is excluded from the result.

6.5. KPI (Key Performance Indicator).

To measure JJBike's realized revenue in relation to monthly targets, the Dimensional.kpi table was created using the 5.KPI.sql file. The script consists of four main structures:

- The SELECT block defines the three columns that will make up the table: dt.time_month aliased as month, which identifies the reference month; SUM(fv.total_price) aliased as total_revenue, which aggregates the revenue actually realized by summing the total_price of all fact_sales records for the respective month; and a CASE block dt.time_month that assigns a fixed target value for each month, from R$ 220,000 in January to R$ 270,000 in December, aliased as target;
- The ::numeric conversion operator, positioned after the END of the CASE statement, causes the target column to be consolidated with the same fixed-precision numeric type as total_price in fact_sales. Without this conversion, PostgreSQL would treat the CASE values as integers, causing type incompatibility when calculating achievement percentages in Power BI;
- The INTO Dimensional.kpi block physically creates the target table in the Dimensional schema, storing the realized revenue and the target side-by-side for each month;
- The FROM block starts from Dimensional.fact_sales fv and performs an INNER JOIN with Dimensional.dim_time dt based on the condition fv.time_key = dt.time_key, relating each fact record to its respective month in dim_time. The GROUP BY dt.time_month consolidates all sales for each month into a single row, and the ORDER BY dt.time_month ASC organizes the result in ascending chronological order.

7. Artifacts – Power BI.

The tables in the Dimensional database, created in pgAdmin 4, were connected to Power BI.

7.1. Reports.

7.1.1. Transformations (T) in the tables.

In fact_sales, a column called discount_percent (with two decimal places) was created with the following formula:

- discount_percent = TRUNC(('dimensional fact_sales'[discount] / 'dimensional fact_sales'[total_price]) * 100, 2);

In dim_customer, a column called location_map was created with the following formula:

- location_map =
  SWITCH(
  'dimensional dim_customer'[customer_state],
  "AC", "Acre",
  "AL", "Alagoas",
  "AM", "Amazonas",
  "AP", "Amapa",
  "BA", "Bahia",
  "CE", "Ceara",
  "DF", "Federal District",
  "ES", "Espirito Santo",
  "GO", "Goias",
  "MA", "Maranhao",
  "MG", "Minas Gerais",
  "MS", "Mato Grosso do Sul",
  "MT", "Mato Grosso",
  "PA", "Para",
  "PB", "Paraiba",
  "PE", "Pernambuco",
  "PI", "Piaui",
  "PR", "Parana",
  "RJ", "Rio de Janeiro",
  "RN", "Rio Grande do Norte",
  "RO", "Rondonia",
  "RR", "Roraima",
  "RS", "Rio Grande do Sul",
  "SC", "Santa Catarina",
  "SE", "Sergipe",
  "SP", "Sao Paulo",
  "TO", "Tocantins",
  "Unknown"
) & ", Brazil".

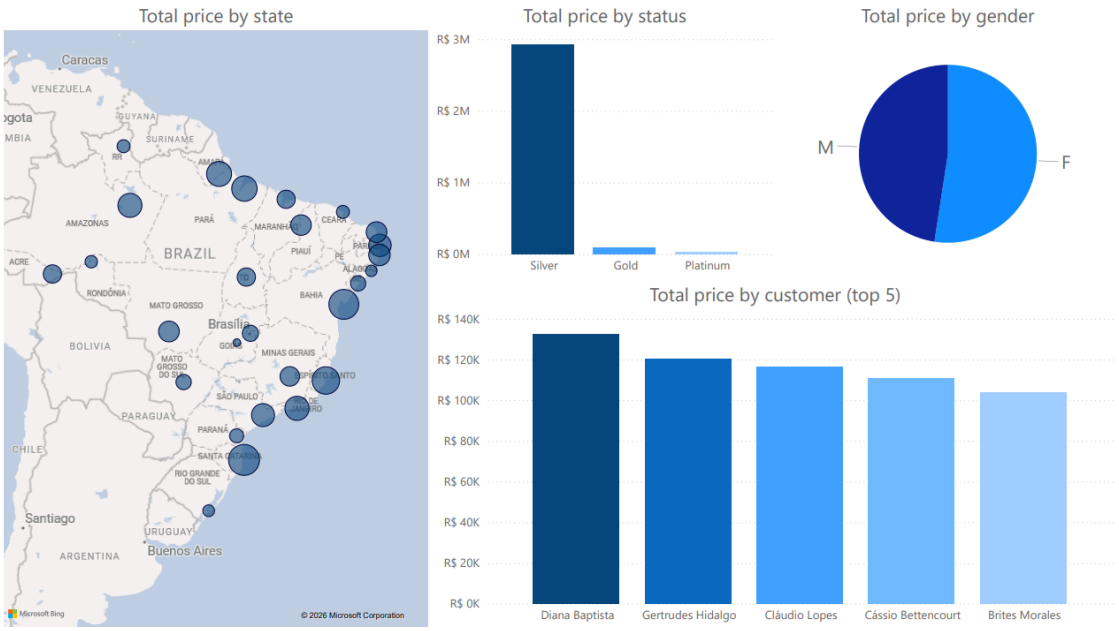
7.1.2. Metrics.

In dim_kpi, three metrics were created so that, in the KPI report, the card has dynamic behavior according to what is selected in the button slicer buttons:

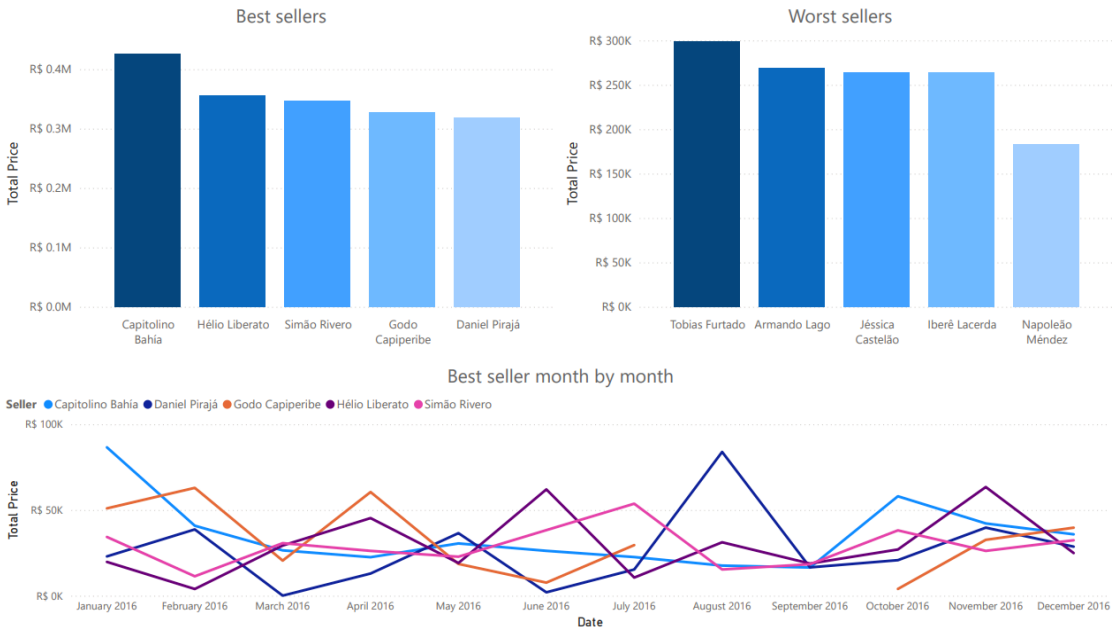
- measure_target =
  IF(
  ISFILTERED('dimensional kpi'[month]) || ISFILTERED('dimensional dim_time'[time_year]),
  SUM('dimensional kpi'[target]),
  0
).
- measure_total_revenue =
  IF(
  ISFILTERED('dimensional kpi'[month]) || ISFILTERED('dimensional dim_time'[time_year]),
  SUM('dimensional kpi'[total_revenue]),
  0
).

7.2.1. Created Reports.

(Customers)
The customer report shows: total value sum by state, total value sum by status, total value sum by gender, and total value sum by customer (top 5).

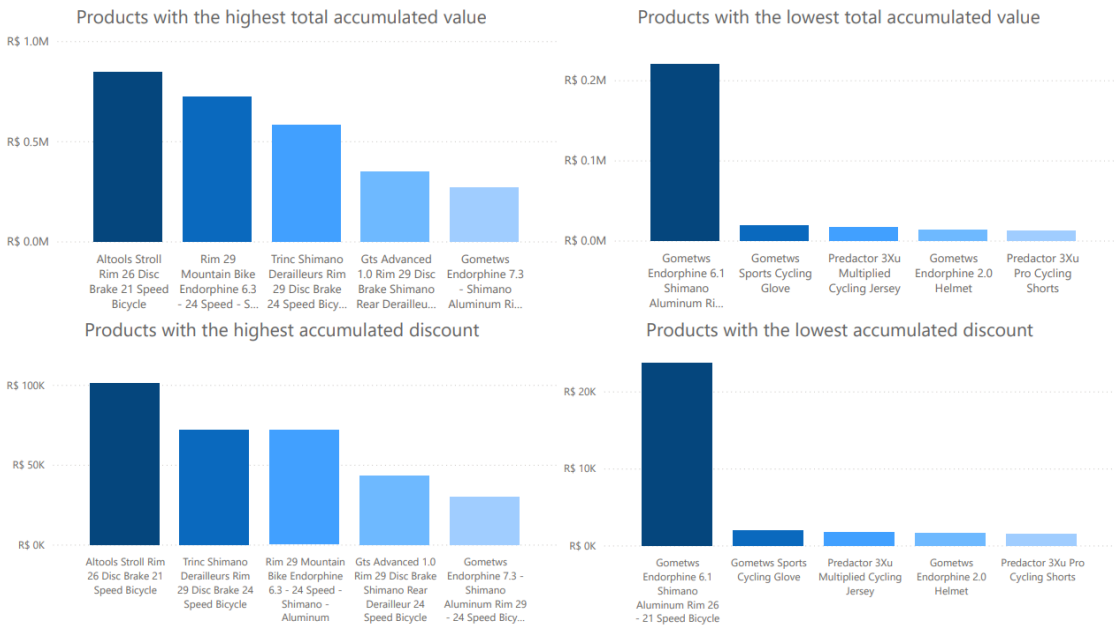


(Sellers)
The sellers report shows: total value sum by salesperson (top 5 best sellers, top 5 worst sellers, and top 5 best sellers compared month by month).



(**Products**)

The product report shows: total value per product (top 5 best-selling products and top 5 least-selling products) and total discount per product (top 5 products with the highest accumulated discount and top 5 products with the lowest accumulated discount).



7.2. Dashboards.

The following dashboards were created:

(Sales)

The sales report shows: total value of products sold (compared month by month), total quantity of products sold (compared month by month), total discount of products sold (compared month by month), and product, sellers, and customer cards for filtering.

(KPI)

The KPI report shows: target, total value acquired, sum of target and total value acquired (compared month by month), and month and year cards for filtering.

7.3. Cube.

7.3.1. Hierarchies created from the dimensions.

The following hierarchies were created:

- Geographic Hierarchy: customer_state → customer_name (dim_customer);
- Date Hierarchy: time_year → time_quarter → time_month → time_day (dim_time);
- Notes:

- customer_gender and customer_status should remain free in the side panel as Loose Dimension Attributes, as inserting them into the Geographic Hierarchy would imply that they are part of a geographic subdivision;
- dim_seller and dim_product have only one attribute in addition to their keys, therefore they did not generate hierarchies and their attributes became Loose Dimension Attributes.

7.3.2. Visual representation of the created cube.

Customer State	Quantity	Discount	Total Price
AC	34	R$ 11,172.65	R$ 88,878.08
AL	6	R$ 4,199.33	R$ 26,817.61
AM	57	R$ 15,881.10	R$ 168,571.45
AP	53	R$ 16,202.60	R$ 183,945.80
BA	65	R$ 18,104.59	R$ 274,690.93
CE	11	R$ 9,146.31	R$ 37,562.17
DF	19	R$ 14,839.02	R$ 65,272.18
ES	89	R$ 12,590.55	R$ 226,925.84
GO	3	R$ 858.20	R$ 7,614.38
MA	38	R$ 5,149.81	R$ 87,431.06
MG	31	R$ 18,853.70	R$ 102,980.97
MS	19	R$ 11,094.30	R$ 57,340.96
MT	44	R$ 11,279.89	R$ 119,272.51
PA	82	R$ 24,476.00	R$ 190,925.91
PB	51	R$ 8,791.16	R$ 143,634.26
PE	42	R$ 19,477.64	R$ 129,473.93
PI	44	R$ 20,851.47	R$ 116,457.99
PR	22	R$ 5,322.31	R$ 45,718.33
RJ	57	R$ 19,318.44	R$ 169,913.05
RN	45	R$ 16,227.19	R$ 118,559.35
RO	13	R$ 6,639.07	R$ 33,213.18
RR	28	R$ 7,962.75	R$ 37,392.82
RS	12	R$ 2,768.23	R$ 28,466.43
SC	91	R$ 35,968.90	R$ 296,803.85
SE	15	R$ 11,054.02	R$ 56,151.97
SP	42	R$ 17,043.43	R$ 152,112.79
TO	47	R$ 4,163.74	R$ 88,034.93
Total	**1060**	**R$ 349,436.40**	**R$ 3,054,162.73**

Generated files available in '2. Artifacts/'.